SharkTooth 論文完整導讀教材

A Scalable Real-Time Algorithm for BLE-Based Wireless Body Sensor Network Synchronization

---

論文出處：IEEE Internet of Things Journal, Vol. 12, No. 22, 15 November 2025, pp. 46174–46192

作者群：Nicolò Landra, Danilo Demarchi, Paolo Motto Ros (Politecnico di Torino, Italy)

教材定位：Meeting 報告、多節點無線感測網路（WBSN）時鐘同步推導、BLE 延遲抑制與工程重現專用。

第 0 部分：閱讀地圖

0.1 三句話核心精要

- 【原文】要解決什麼問題： 無線人體感測網路（WBSN）在健康醫療與動作分析中高度依賴多感測器時間同步，然而低功耗藍牙（BLE）連接模式受限於底層晶體振盪器頻率漂移、不可預測的封包重傳與作業系統回調延遲，造成嚴重時鐘不同步，而既有方法多依賴硬體修改或廣播模式（Advertising mode），無法在標準硬體上實現高擴展性的即時雙向數據傳輸同步（p.46174–46176）。
- 【原文】為什麼重要： 若時間未同步，多節點的肌電（sEMG）與慣性（IMU）訊號融合將產生嚴重相位錯位與假性因果推斷；若能完全在「應用層（Application Layer）」解決同步問題，無需客製化韌體或硬體時間戳，即可讓現成商用（COTS）BLE 設備無縫互通，推動穿戴式物聯網生態系普及（p.46174, p.46189）。
- 【原文】用什麼方法解決： 本文提出 SharkTooth 演算法，完全在主機應用層運作，透過建立時間延遲差值（δT）的下界幾何模型，利用「樞紐點（Pivot Point）」動態偵測結合「自適應動態閾值（δT_{thr}）」與「滑動方差權重（λ）」，濾除所有重傳突波，僅抓取貼近線性漂移底線的有效封包，並以閉式遞迴普通最小二乘法（Recursive OLS）即時校準時鐘，達成 12 個感測節點下 0.21–0.62 ms 的亞毫秒級即時同步精度（p.46178–46182）。

0.2 主要論文貢獻（對應原文宣稱之 Contributions）

1. 【原文】首個純應用層、無修改韌體的即時連線同步架構：徹底擺脫對 MAC 層微秒級硬體計時器或晶片專屬周邊（如 Nordic PPI）的依賴，直接在標準 BLE 連接模式的 GATT Notification 數據封包上實現雙向同步，相容於所有商用 COTS BLE 感測設備（p.46176, p.46180）。
2. 【原文】非線性隨機延遲抑制技術（**SharkTooth** 核心算法）：提出基於動態幾何探針與延遲差值下邊界（Lower Bound）的自適應校正機制，能智慧化識別並剔除由封包重傳（Retransmission）與多執行緒軟體抖動引起的非確定性階躍突波（p.46176, p.46180–46181）。
3. 【原文】超高擴展性驗證（支援至 **12** 個節點）：打破過去連線模式研究僅驗證 2～4 個節點的瓶頸，透過多 Dongle 拓撲架構在 18 種網路配置、涵蓋 17 至 244 Bytes 封包長度下進行每組 20 次、每次 1 小時的嚴苛長時測試，驗證系統在重度網路擁塞下的強健收斂性（p.46176, p.46182–46186）。
4. 【原文】同步誤差與網路吞吐量之定量規律建模：首次在真實環境中量化了 BLE 網路總吞吐量（最高達 30 kB/s）與同步誤差（95th 百分位數）之間的線性正比關係，為生物醫學工程師設計多通道肌電（2 kB/s/節點）與全身 IMU 動作捕捉系統提供了精確的理論設計依據（p.46189–46190）。

0.3 整個系統的方法流程圖

```
==============================【1. 實體硬體與 BLE 封包傳輸 (Hardware & BLE Stack)】==============================
周邊節點 (Peripherals 1~12: Apollo3 Blue MCU) ──> 類比前端採樣 (sEMG / IMU, 50 Hz 測試訊號)
  │ (ADC 中斷觸發立即打上本地時間戳 t_P)
  ▼
[構建 BLE Notification 封包 (最大 244 Bytes)]:
  ┌──────────────────────────────────────────┬──────────────────────┬──────────────────┐
  │ 感測數據樣本 (Up to 239 Bytes)   │ 本地時間戳 t_P (4B) │ 封包序號 ID (1B) │
  └──────────────────────────────────────────┴──────────────────────┴──────────────────┘
  │ (經由 BLE 4.2 連接模式傳輸，連接間隔 CI = 30 ms)
  ▼ [無線通道：遭遇排程衝突、封包遺失、重傳延遲 (造成 k × CI 隨機跳動)]
中央接收器 (Central: 3 支 nRF52840 USB Dongles, 各分擔 4 個節點)
  │ (觸發 USB 虛擬串口 Serial Callback)
  ▼
控制平台應用層 (Python Control Platform) ──> 接收時刻打上中央系統時間戳 t_C
  │
==============================【2. SharkTooth 核心校正演算法 (Algorithm 1 & 2)】==============================
新封包到達：提取配對時間戳組 (t_P^i, t_C^i)
  │
  ▼ [1. 計算觀測時延與二階差分 (Eq. 3, 5)]
```


δT^i = t_C^i - t_P^i
δ^2T^i = δT^i - δT^(i-1)
▼ [2. 計算近 100 點滑動標準差與變異度權重 (Eq. 7, 10)]
σ_100^i = StdDev(δ^2T)
λ_0^i = min(CI / (2 * σ_100^i), 1) (表徵通訊品質，噪訊大時 λ_0 趨近 0)
【情況 A:發現全新全域最小值 (樞紐點 Pivot Point)】
▼ (Eq. 11)
[更新樞紐點 (t_P0, δT_0) = (t_P^i, δT^i)]
探針斜率重設 γ_1 = 10^(-8)，模型斜率 γ_0 = 0
啟動 30 秒暫態沉降冷卻期 (Settling Period)
重設遞迴回歸矩陣 A^i = 0, b^i = 0 (Eq. 18)
【情況 B:δT^i ≥ δT_0 (常態運作)】
▼
[比較幾何動態探針門檻 (Eq. 9)]
δT_thr^i = δT_1 + γ_1 * (t_P^i - t_P1)
是否符合資格 (δT^i < δT_thr^i)？
【是:合格邊界點 (Eq. 12, 13)】
平滑更新模型斜率 γ_0 (加權平均)
探針重設:γ_1 = 10^(-8), (t_P1, δT_1) = (t_P^i, δT^i)
【否:重傳延遲突波 (Eq. 14)】
判定為異常值，拋棄該點！
探針斜率微增:γ_1 = γ_1 + 10^(-6) (防止脫靶)
▼
▼ [3. 輸出下界校正延遲 (Eq. 8)]
δT_*^i = δT_0 + γ_0 * (t_P^i - t_P0)
▼ [4. 遞迴普通最小二乘法更新 (Recursive OLS, Eq. 15–17)] (冷卻期 30 秒過後)
秩一矩陣更新:A^i = A^(i-1) + [1, -t_P; -t_P, (t_P)^2], b^i = b^(i-1) + [t_P*δT_*; δT_*]
即時求解時鐘偏移與漂移參數:[α^i, β^i]^T = (A^i)^(-1) * b^i
▼ [5. 計算最佳估計延遲與最終同調時間戳 (Eq. 19, 20)]
δT̂^i = α^i * t_P^i + β^i
t̂_S^i = t_P^i + δT̂^i
==========【3. 輸出與多感測器資料融合】==========
所有 12 個節點之數據流統一轉換至全域時間軸 t̂_S (相對誤差 RSE 穩定落於 0.21–0.62 ms 區間)

0.4 建議閱讀與報告準備順序

- 第一優先（**Meeting 報告必講精華**）：
  1. **§I-A & §III-A**（延遲物理模型）：用 Fig. 3 與 Fig. 4 說明 BLE 連線模式下 δT 的雙重本質——乾淨環境下的「鋸齒波（SharkTooth）」與雜訊環境下的「重傳階躍突波」（p.46178–46179）。
  2. **§III-C**（演算法幾何直覺）：報告的核心靈魂！解釋為什麼一般線性回歸會被重傳突波拉跑，而 SharkTooth 如何利用樞紐點（Pivot）與動態探針門檻（δT_{thr}）死死咬住下邊界（Fig. 6, p.46180–46181）。
  3. **§V-B & Table III**（與 **SOTA** 橫向對比）：突顯本研究相較於先前文獻（Li et al., Balasubramanian et al., Depolli et al.）在節點數（2➔12）、即時性（Online vs. Offline）與純應用層實現上的壓倒性優勢（p.46187–46189）。
  4. **§V-C & Fig. 13**（吞吐量與生醫場景）：連結生醫應用！說明 2 kB/s 的 sEMG 與 2.5 kB/s 的 IMU 在網路達到 30 kB/s 極限時，同步誤差如何保持在取樣週期以下（p.46189–46190）。
- 第二優先（實驗重現必讀數學與程式）：
  1. **Algorithm 1 & Algorithm 2**（虛擬碼流程，**p.46180**）：逐行搞懂狀態機轉移條件。
  2. **Eq. (10)** 至 **Eq. (14)**（自適應加權與探針步進）：弄清楚 λ_0 的除法限制、λ 的條件常數 0.5、以及 10^{-6} 步進係數的物理意涵。
  3. **Eq. (15)** 至 **Eq. (18)**（遞迴 **OLS** 閉式解與 **30** 秒冷卻重設）：理解為何複雜度只有 $O(1)$，適合邊緣嵌入式或 Python 背景執行緒。
- 可快速略讀：
  1. §II-A（硬體電路板具體尺寸與電源接腳細節，p.46176–46177）。
  2. §IV-A（示波器型號與麵包板跳線物理連接描述，p.46182–46183）。

第 1 部分：先備背景知識

1.1 晶體振盪器（Crystal Oscillator）的硬體物理漂移

- 【直覺解釋】每個微控制器（MCU）內部都有一個石英震盪器（如同心臟跳動）來計算時間。但在製造工藝上，沒有任何兩顆石英晶體的物理切割角度是完全相同的。標稱 32.768 kHz 的晶體，實際跑起來可能是 32.7685 kHz 或 32.7678 kHz。
- 【正式定義】此誤差稱為頻率容差（Frequency Tolerance），以百萬分之一（parts per million, ppm）表示。例如 $\pm\ 20ppm$ 代表每經過 1,000,000 秒，時鐘最多會累積快或慢 20 秒；折算下來，每小時會累積高達 $72ms$ 的線性時間漂移！ 若不進行動態補償，不同感測節點的波形在短短幾分鐘內就會完全錯位。

1.2 低功耗藍牙（BLE）通訊架構：廣播 vs. 連線模式

- 【廣播模式（Advertising Mode）】節點單向廣播資料，中央端只聽不回。優點是傳輸無交握、延遲確定性高（過去高精度同步如 BlueSync, CheepSync 均基於此）；致命缺點是**完全不可靠（掉包無法重傳）**、無法雙向控制設備、且單一封包資料酬載極小。
- 【連線模式（Connection Mode）】中央（Central）與周邊（Peripheral）建立皮網（Piconet），依據固定的連線間隔（Connection Interval, CI，例如 30 ms）輪流排程通訊。優點是具有 ACK 確認機制與自動重傳，能安全傳輸連續大數據；致命缺點是重傳會導致到達時間產生 **$1\times\text{CI}, 2\times\text{CI}$** 的整數倍跳躍階躍延遲，這正是傳統線性回歸同步演算法失效的元凶。

1.3 為什麼傳統回歸（如 FTSP, Li et al.）在多節點連線模式下會崩潰？

- 【直覺解釋】普通線性回歸（OLS）基於「高斯白雜訊（均值為 0、變異數固定）」的假設。但在 BLE 連線模式下，重傳引起的延遲**永遠是正向增加的離群值（One-sided Outliers）**。如果直接對所有到達封包做回歸，那些被重傳延遲拖慢的封包會像槓桿一樣把回歸直線向上硬生生拉歪，導致斜率估計嚴重失真。
- 【下邊界哲學（Lower-Bound Principle）】在所有接收到的封包中，延遲最小的封包必然是「零重傳、無排程阻塞、以光速與純硬體最小路徑傳遞」的幸運封包。這些最小延遲點所構成的下邊界，才真實反映了晶體振盪器的物理線性漂移！SharkTooth 的核心哲學就是**「只抓下邊界，其餘全部視為雜訊剔除」**。

第 2 部分：逐節導讀（平行對照）

---

§I. INTRODUCTION（引言，p.46174–46176）

- 本節要回答的問題：為什麼 WBSN 必須達到亞毫秒級同步？BLE 上的主要瓶頸為何？
- 關鍵句子解釋：【原文】"A commonly accepted threshold is achieving synchronization accuracy within the sampling interval, ensuring sample-level precision." (p.46174)【白話】同步精度的黃金準則是「誤差必須小於採樣週期」。對於 1 kHz 採樣的肌電訊號，採樣週期為 1 ms；對於 200 Hz 的 IMU，採樣週期為 5 ms。因此，同步演算法必須跨越 1 ms 大關，才能確保訊號在採樣點級別（Sample-level）完美對齊。
- **Checkpoint**：過去的高精度同步若非走 MAC 層硬體時間戳（無法跨硬體移植），就是走廣播模式（無法做雙向控制）。本篇立志在「純應用層」啃下連線模式這塊硬骨頭。

§II. SYSTEM DESCRIPTION（系統描述，p.46176–46177）

- 本節要回答的問題：本實驗採用何種硬體與軟體架構來驗證演算法？
- 關鍵規格：
  - 節點硬體：SparkFun Artemis 模組（搭載 Ambiq Apollo3 Blue ARM Cortex-M4F MCU），外掛 32.768 kHz 石英晶體（容差 $\pm\ 20ppm$，做為本地時間戳 t_P 來源）。
  - 感測前端：sEMG 差動類比前端（放大倍率 200–1600，頻寬 30–400 Hz）+ ST LSM6DSO32 6 軸 IMU。
  - 控制平台：Python 3，搭配 Nordic nRF52840 USB Dongle（運行 Blatann 庫）做為中央接收器。

§III. SharkTooth ALGORITHM（SharkTooth 演算法，p.46177–46182）

- 本節要回答的問題：如何在數學上建模時間戳延遲？如何用自適應幾何演算法剔除重傳突波？
- 關鍵概念演進：
  1. **§III-A**（時鐘模型）：建立 $\delta T = \alpha t_P + \beta + \epsilon(t_S)$，指出 δT 包含常態鋸齒狀（CI 週期內隨機相位）與重傳突波（大於 CI）。
  2. **§III-B**（通訊協定）：封包攜帶 4B 本地時間戳 t_P 與 1B 序號，中央接收時以內部計時器賦予 t_C。
  3. **§III-C**（核心演算法）：引入樞紐點、動態探針門檻、滑動標準差懲罰與遞迴 OLS 更新（詳見第 3 部分精講）。

§IV. VALIDATION（實驗驗證，p.46182–46185）

- 本節要回答的問題：如何在沒有昂貴外部微秒級儀器的情況下，客觀評估 12 個無線節點的真實同步誤差？

- 巧妙的 **Ground-Truth** 驗證設計：【原文】利用 Rigol 示波器同時輸出兩組反相位的 5 Hz 正弦波電壓，兵分 12 路注入所有節點的 sEMG 差動放大器。節點以 50 Hz 採樣。因為所有節點物理上接收到的是同一個連續正弦波，透過**互相關（Cross-Correlation, CC）**將訊號升取樣至 10 kHz，即可提取出物理真實的相對延遲 τ_{nm}，並與 SharkTooth 估計的相對時間差 $\hat{\tau}_{nm}$ 相減得到相對同步誤差（RSE）（p.46183–46185）。

§V. RESULTS AND DISCUSSION（結果與討論，p.46185–46190）

- 本節要回答的問題：演算法在不同節點數、封包長度與長時運行下的表現如何？
- 核心成果：
  - 在最簡單配置（A2：2 節點，17 Bytes）下，中位絕對平均誤差僅 **0.21 ms**。
  - 在最嚴苛極限配置（C12：12 節點，244 Bytes 最大封包）下，前期誤差 1.27 ms，隨時間自適應收斂至 **0.62 ms**（Table II）。
  - Fig. 12 證實封包間隔方差 σ^2_{100} 與同步誤差高度正相關；當方差壓在臨界值 $225ms^2$（即 $(CI/2)^2$）以內時，誤差極度平穩。

第 3 部分：數學逐式精講

Eq. (1) & (2)／p.46178 —— 理想時鐘與中央觀測時鐘模型

$$t_S = (\alpha + 1)t_P + \beta$$

$$t_C = t_S + \epsilon(t_S)$$

- 符號表：
  - $t_S \in R^+$：連續全域參考基準時間（True Reference Sampling Time），物理世界的絕對標準時間。
  - $t_P \in R^+$：周邊節點本地晶體振盪器記錄的時間戳（Peripheral Timestamp）。已知量（由封包提供）。
  - $t_C \in R^+$：中央主機在接收到封包回調瞬間賦予的系統時間戳（Central Timestamp）。已知量。
  - $\alpha \in R$：周邊相對於中央的頻率漂移係數（Clock Drift Coefficient，量級通常在 $\pm\ 20ppm = \pm\ 20 \times 10^{-6}$）。待估計參數。
  - $\beta \in R$：周邊與中央啟動計時器的初始時間相位差（Clock Offset Coefficient）。待估計參數。
  - $\epsilon(t_S) \geq 0$：封包傳輸、協定堆疊排程與作業系統回調引起的單向隨機通訊延遲。
- 白話解釋：Eq. (1) 描述了周邊節點的本地時鐘與絕對時間之間的線性映射；Eq. (2) 描述了中央主機收到的時間等於絕對時間加上一段不可避免的正向隨機延遲。

Eq. (3) & (4)／p.46178 —— 觀測時延差值展開式

$$\delta T = t_C - t_P$$

$$\delta T = \alpha t_P + \beta + \epsilon(t_S)$$

推導過程：將 Eq. (1) 代入 Eq. (2)：

$$t_C = \big((\alpha + 1)t_P + \beta\big) + \epsilon(t_S) = \alpha t_P + t_P + \beta + \epsilon(t_S)$$

兩邊同時減去 t_P：

$$\delta T = t_C - t_P = \alpha t_P + \beta + \epsilon(t_S)$$

-
- 物理意義解析：量測到的時延差 δT 由三部分疊加而成：1. αt_P：隨時間線性傾斜的漂移項（直線斜率）。2. β：恆定的初始截距項。3. $\epsilon(t_S)$：上下跳動的隨機擾動。在理想無重傳情況下，$\epsilon(t_S)$ 在一個連線間隔（$CI = 30ms$）內週期性波動，形如鯊魚牙齒（SharkTooth，見 Fig. 3b）；在重傳情況下，$\epsilon(t_S)$ 會爆衝至數十甚至數百毫秒（Fig. 4b）。

Eq. (5) & (6)／p.46178 —— 二階封包間隔時延差分（Inter-Packet Delay Difference）

$$\delta^2 T^i = \delta T^i - \delta T^{i-1}$$

$$\delta^2 T^i = \alpha(t_P^i - t_P^{i-1}) + \epsilon(t_S^i) - \epsilon(t_S^{i-1})$$

- 符號表：上標 i 代表第 i 個依序到達的 BLE 數據封包。
- 推導與白話解釋：相鄰兩次採樣之間，周邊時間步進 $(t_P^i - t_P^{i-1}) \approx \Delta t$ 為常數，而漂移量 $\alpha \approx 10^{-5}$ 極小，因此第一項 $\alpha(t_P^i - t_P^{i-1}) \approx 0$。這意味著二階差分 $\delta^2 T^i \approx \epsilon(t_S^i) - \epsilon(t_S^{i-1})$，它徹底過濾掉了時鐘漂移的直流趨勢，純粹反映了通訊通道延遲的劇烈抖動程度！

Eq. (7)／p.46178 —— 滑動視窗標準差（Moving Standard Deviation）

$$\sigma_M^i = \sqrt{\frac{1}{M} \sum_{n=i-M}^{i} (\delta^2 T^n - \mu_M^i)^2}$$

- 符號表：M 為滑動視窗長度（論文設為 $M = 100$ 個封包，在 10 Hz 傳輸下對應最近 10 秒）。μ_M^i 為近 M 點之二階差分均值。
- 物理意義：σ_{100} 是網路通訊品質的晴雨表。在完全無掉包的理想通道下，延遲在 $[0, CI]$ 間均勻波動，差分變異數有理論極限；一旦發生掉包重傳，σ_{100} 會驟然飆升（Fig. 4c）。

Eq. (8)／p.46180 —— 鯊魚齒下界校正延遲模型（Output Delay Equation）

$$\delta T_*^i = \delta T_0 + \gamma_0 (t_P^i - t_{P0})$$

- 符號表：
  - δT_*^i：經演算法校正後的「下界估計延遲值」（圖 6 中的紅色切線）。
  - $(t_{P0}, \delta T_0)$：樞紐點（Pivot Point）座標，即目前為止觀測到的 δT 歷史全域最小值。固定錨點。
  - γ_0：下界切線的估計斜率。自適應更新變數。
- 白話解釋：任何真實的通訊延遲都不可能小於物理極限。因此歷史上最小的 δT 點最接近無延遲真實值。這條線以樞紐點為起點，以斜率 γ_0 向前延伸，死死貼在所有散佈點的最底層邊緣。

Eq. (9)／p.46181 —— 動態幾何探針門檻（Dynamic Probe Threshold）

$$\delta T_{thr}^i = \delta T_1 + \gamma_1 (t_P^i - t_{P1})$$

- 符號表：

- δT^{i}_{thr}：動態探針閾值。用來判定新進封包是否具備「更新資格」。
- $(t_{P1}, \delta T_1)$：上一次被判定為合格點時的座標。
- γ_1：探針門檻的張開斜率（Research Coefficient）。初始值設為極小的 10^{-8}。

- 白話解釋：這是一條從上一個合格點出發、逐漸向上抬起的「搜索光束」。如果新來的點落在光束下方，代表它非常貼近下邊界，准予接納；若落在光束上方，代表是重傳延遲，直接拋棄。

Eq. (10)／p.46181 —— 通道品質歸一化係數

$$\lambda_0^i = \min\left(\frac{CI}{2\sigma_M^i}, 1\right)$$

- 極端檢查：
  - 當通訊完美無重傳時：$\sigma_{100} \approx \frac{\text{CI}}{2} = 15\text{ ms} \implies \frac{\text{CI}}{2\sigma_{100}} \approx 1 \implies \lambda_0 = 1$。
  - 當網路擁塞、頻繁重傳時：$\sigma_{100} \gg 15\text{ ms} \implies \frac{\text{CI}}{2\sigma_{100}} \to 0 \implies \lambda_0 \to 0$。
- 作用：作為更新權重，當網路噪訊大時自動將更新步長降為 0，凍結模型，避免被污染。

Eq. (11)／p.46181 —— 樞紐點重設機制（Pivot Point Identification）

$$\delta T^i < \delta T_0 \implies \begin{cases} \delta T_0 = \delta T^i \\ \gamma_0 = 0 \\ t_{P0} = t_P^i \\ \gamma_1 = 10^{-8} \end{cases}$$

- 白話解釋：只要看到一個比歷史最小值更小的延遲點，代表找到了更乾淨的「絕對物理下界錨點」，立刻將該點設為全新樞紐點，並將斜率歸零重置。

Eq. (12) & (13)／p.46181 —— 自適應斜率平滑更新與權重條件式

$$\gamma_0 = (1 - \lambda^i)\gamma_0 + \lambda^i \frac{\delta T^i - \delta T_0}{t_P^i - t_{P0}}$$

$$\lambda^i = \begin{cases} 0.5 \lambda_0^i, & \delta T^i \ge \delta T_*^{i-1} \\ 0.5, & \delta T^i < \delta T_*^{i-1} \end{cases}$$

- 白話解釋：當新點合格時，它與樞紐點連線的斜率為 $\frac{\delta T^i - \delta T_0}{t_P^i - t_{P0}}$。我們用指數加權移動平均（EWMA）更新模型斜率 γ_0。權重 λ^i 的設計極具巧思：
  - 若新點落在當前下界線上方（$\delta T^i \ge \delta T_*^{i-1}$）：更新權重縮減為 $0.5\lambda_0^i$，高度依賴通訊品質，謹慎慢行。
  - 若新點落在當前下界線下方（$\delta T^i < \delta T_*^{i-1}$）：說明當前下界估高了，必須迅速下修！強制將權重拉到最大上限 0.5，快速壓制下界線貼齊新點。

Eq. (14)／p.46181 —— 探針搜索斜率主動遞增（Threshold Adaptation）

$$\delta T^i \ge \delta T_{thr}^i \implies \gamma_1 = \gamma_1 + 10^{-6}$$

- 白話解釋：如果連續多次都沒有新點落入門檻內，探針會將自己的斜率每次增加 10^{-6}（量級匹配於石英晶體漂移率 ppm）。這確保探針會「慢慢抬頭」，防止因時鐘自然向上漂移而導致後續點永遠被擋在門外的死鎖現象（Deadlock）。

Eq. (15), (16), (17)／p.46182 —— 遞迴普通最小二乘法（Recursive OLS Formulation）

$$\begin{bmatrix} \alpha^i \\ \beta^i \end{bmatrix} = \frac{1}{\det(A^i)} A^i b^i, \quad \text{if } \det(A^i) \ne 0$$

$$A^i = A^{i-1} + \begin{bmatrix} 1 & -t_P^i \\ -t_P^i & (t_P^i)^2 \end{bmatrix}, \quad b^i = b^{i-1} + \begin{bmatrix} t_P^i \delta T_*^i \\ \delta T_*^i \end{bmatrix}$$

- 數學本質還原：【補足推導】傳統二維線性擬合目標為 $y = \alpha x + \beta$，正規方程式（Normal Equations）為：$$\begin{bmatrix} \sum (t_P^i)^2 & \sum t_P^i \\ \sum t_P^i & \sum 1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum t_P^i \delta T_*^i \\ \sum \delta T_*^i \end{bmatrix}$$ 設設計矩陣協方差為 $M = \begin{bmatrix} \sum (t_P^i)^2 & \sum t_P^i \\ \sum t_P^i & N \end{bmatrix}$，其伴隨矩陣（Adjugate Matrix）恰為：$$\text{adj}(M) = \begin{bmatrix} N & -\sum t_P^i \\ -\sum t_P^i & \sum (t_P^i)^2 \end{bmatrix}$$ 論文中 Eq. (16) 的矩陣 A^i 實質上就是直接維護此**伴隨矩陣的增量形式**！因此 $\det(A^i) = \det(M)$，且 $M^{-1} = \frac{1}{\det(A^i)} A^i$。透過秩一（Rank-one）累積，每次只需加法與一個 2×2 矩陣乘法，時間與空間複雜度均為嚴格的 $O(1)$！

Eq. (18)／p.46182 —— 30 秒沉降冷卻期（Settling Period Reset）

$$t_P^i \le t_{P0} + 30\text{ s} \implies \alpha^i = 0, \; \beta^i = 0, \; A^i = 0_{2\times 2}, \; b^i = 0_{2\times 1}$$

- 白話解釋：樞紐點剛更新時，斜率 γ_0 與下界線尚處於初期震盪過渡期（見 Fig. 6 前 30 秒）。在此期間強制將回歸矩陣清零，不急著估計 α, β，待 30 秒沉降穩定後再開始累積回歸矩陣，徹底消除啟動暫態噪訊。

Eq. (19) & (20)／p.46182 —— 最終同調同步時間戳生成

$$\delta \hat{T}^i = \alpha^i t_P^i + \beta^i$$

$$\hat{t}_S^i = t_P^i + \hat{\delta T}^i$$

- 白話解釋：利用收斂的 α, β 計算出目前該周邊的最佳無延遲時延補償量 $\hat{\delta T}^i$，加回本地時間戳 t_P^i，即獲得對齊到微秒級的全域同步時間戳 $\hat{t}_S^i$！

第 4 部分：圖表解讀

Figure 3／p.46179 —— 理想通訊環境下的「鯊魚齒（SharkTooth）」波形

- **(a)** 全域視圖：δT 隨 t_P 呈現一條極度狹窄、微微向上傾斜的藍色直線（反映 α 晶體線性漂移）。
- **(b)** 局部放大視圖（核心特色）：在 5 秒視窗內，δT 呈現週期性上下鋸齒狀起伏，波峰與波谷之差嚴格落在一個連線間隔 $CI = 30ms$ 之內！這是因為周邊採樣（100 ms）與 BLE 連線事件（30 ms）之間存在恆定的外差拍頻（Beat Frequency）。
- **(c)** 滑動標準差 σ_{100}：數值極度穩定地在 $CI/2 = 14.5ms$ 上下微幅波動。

Figure 4／p.46179 —— 吵雜通訊環境下的重傳突波

- **(a)** 全域視圖：鋸齒線上方佈滿了大量向上竄升的毛刺與階躍突波（高達 1.35～1.40 秒）。
- **(b)** 局部放大視圖：規律的鋸齒波被稀疏但巨大的重傳延遲打斷。
- **(c)** 滑動標準差 σ_{100}：數值從 14 ms 暴增至 30～50 ms，清晰辨識出通訊品質的劣化。

Figure 6／p.46181 —— SharkTooth 演算法消除非線性延遲動態過程（必講重點）

- 視覺對比：藍色散佈點為原始帶有大量重傳雜訊的 δT；綠色星號為檢測到的樞紐點（Pivot）；紅色粗線為演算法輸出的校正下界線 δT_*。
- 震撼結論：紅線在前 10 秒內迅速從上方下潛收斂，隨後以近乎完美的直線緊貼在所有散佈點的最底層邊緣，完全無視上方數百個高達 0.35～0.40 秒的干擾雜訊點！

Figure 9／p.46184 —— 同步前 vs. 同步後的 10 節點訊號相位對齊實測

- 上圖（**Synchronization OFF**）：10 顆節點各自以本地時間戳 t_P 繪製同一個 5 Hz 正弦波，相位一片混亂，各走各的路。
- 下圖（**Synchronization ON**）：經過 SharkTooth 輸出之 $\hat{t}_S$ 校正後，10 條正弦波波形**100% 完美重疊融合成單一條平滑曲線**！視覺震撼力極強。

Table II／p.46186 —— 18 種配置在 6 個 600 秒時間段的 RSE 統計表

配置 ID	封包大小	周邊節點數	第 1 段 (0–600s) 中位數 (IQR)	第 6 段 (3000–3600s) 中位數 (IQR)	標準差變化 (1 段➔6段)
A2	17 B	2	0.30 ms (0.08)	0.21 ms (0.02)	0.33 ➔ 0.25 ms
A6	17 B	6	0.45 ms (0.13)	0.34 ms (0.10)	0.50 ➔ 0.25 ms
A12	17 B	12	0.57 ms (0.22)	0.41 ms (0.15)	0.59 ➔ 0.25 ms
B12	127 B	12	0.82 ms	0.50 ms	0.94 ➔ 0.30

配置 ID	封包大小	周邊節點數	第 1 段 (0–600s) 中位數 (IQR)	第 6 段 (3000–3600s) 中位數 (IQR)	標準差變化 (1 段➔6段)
			(0.43)	(0.21)	ms
C12	244 B	12	1.27 ms (0.46)	0.62 ms (0.46)	1.57 ➔ 0.41 ms

- 分析結論：節點越多、封包越大，初期誤差越大（網路擁塞導致掉包）；但隨著時間推移，所有 18 種配置的標準差全部腰斬至 0.25～0.41 ms，最終誤差全數收斂至 1 ms 目標線以下！

Figure 12／p.46188 —— 95th 百分位 RSE 與封包差分方差 σ^2_{100} 之線性關係

- 關鍵臨界線：紅色虛線標記 $\sigma^2_{100} = 225ms^2$（即 $(CI/2)^2$）。所有落在此線左側的點，RSE 均小於 1 ms；落在右側者，RSE 隨方差線性上升。
- 時間收斂：從 Segment 1 到 Segment 6，迴歸黑線斜率顯著變平，點群向左側聚集，證明時間拉長會自發平均掉隨機抖動。

Table III／p.46189 —— 與現有 SOTA 同步技術橫向對比

- 節點規模：本作支援 2➔12 節點（先前連線模式文獻如 [8] 僅 4 節點，[25] 僅 2 節點，[6] 僅 3 節點）。
- 運作模式：本作為唯一兼具「連線模式（Connection）」、「在線即時（Online）」、「隨感測數據封包附帶同步（Sync via Data Packets）」且「精度達 0.21–0.62 ms」之演算法。

第 5 部分：實驗設計與評估

5.1 測試矩陣設計（18 種配置 × 20 次重複）

- 配置維度：節點數（2, 4, 6, 8, 10, 12）× 封包負載（17 B 基本、127 B 中等、244 B 最大上限）= 18 種排列組合。
- 重複次數：每種組合執行 20 次獨立試驗，每次長達 3,600 秒（1 小時），共計 360 小時的巨量實測數據，統計嚴密性極高。
- 網路拓撲技巧：12 顆節點若全塞給單一 Dongle，會強迫 CI 放大至 90 ms（無法傳輸 1 kHz 肌電）；作者拆分為 3 支 nRF52840 Dongle（每支負責 4 節點），既保證了 CI = 30 ms，又完整模擬了 3 個 Piconet 近距離共存時的同頻通道碰撞挑戰。

5.2 評估指標：相對同步誤差（RSE）

$$RSE_{nm}^{k} = \hat{\tau}_{nm}^{k} - \tau_{nm}^{k}$$

- $\hat{\tau}_{nm}^{k} = \hat{t}_{Sn}^{k} - \hat{t}_{Sm}^{k}$：演算法在第 k 秒內估算之節點 n 與 m 的相對時間差。
- τ_{nm}^{k}：升取樣至 10 kHz 後，以互相關（CC）最大值求得之物理真實正弦波相位時差。
- 最劣節點對選取（**Worst-case Pair**）：取每秒內所有節點對中 95th 百分位數最大者作為 $RSE_{\max}^{k}$（Eq. 27），確保評估標準不被平均值粉飾太平。

第 6 部分：批判性閱讀與重現指南

6.1 隱含假設與現實局限

- 振盪器漂移短期恆定假設：演算法假設時鐘漂移 α 隨時間緩慢線性變化。若感測節點經歷急劇劇烈的溫度衝擊（例如從冷氣房走入酷暑戶外），晶體頻率會發生二階非線性跳變，OLS 回歸矩陣可能需要加入遺忘因子（Forgetting Factor）。
- 下界不可穿透假設：演算法強烈依賴「重傳只會增加延遲，不會縮短延遲」。但在多執行緒 OS 中，若 Central 端時間戳被打在作業系統極端排程延遲的回調中，可能偶爾出現假性偏小的異常點（雖然機率極低）。

6.2 審稿人可能提出的質疑與防禦

- 質疑 **1**：為什麼要用 Python 寫在電腦端？這樣不是引入更多作業系統抖動嗎？

  作者回應（**p.46177**）：這正是本研究的刻意設計！如果連在高度不確定的 Python 與通用 OS 排程下都能達到 0.5 ms 精度，證明此演算法具有極強的強健性，能抵抗上層堆疊的隨機延遲。
- 質疑 **2**：初期 30 秒沉降期內，系統完全沒有同步輸出，這在某些隨開即用的系統中能接受嗎？

  作者回應：30 秒僅佔數小時臨床監測的極小比例；若需即時啟動，可直接暫時退化為以 δT_*^i 作為一階近似值，待 OLS 收斂後平滑切換。

6.3 重現時的工程大坑（**Pitfalls**）

1. 探針步進係數 10^{-6} 的時間尺度匹配：論文中的 $+\ 10^{-6}$ 是基於封包發送頻率為 10 Hz（每秒 10 個點）。如果你的系統發送頻率改為 50 Hz 或 1 Hz，必須將此步進值依比例縮放，否則探針抬頭太快會吃進雜訊，抬頭太慢會鎖死。
2. 數值溢位與整數時間戳：節點 t_P 常用 32-bit 無符號整數（微秒或滴答計數器），在長時間運行時會發生溢位翻轉（Rollover）。在進入 SharkTooth 之前，必須在接收端實作時間戳展開（Unwrap）邏輯。
3. 矩陣行列式為 **0** 的除零保護：在剛啟動或所有 t_P 點完全重疊時，$\det(A^i) = 0$。程式碼中必須強制加入 if abs(det) > 1e-12 條件判斷，否則會直接觸發例外崩潰。

第 7 部分：實作對照（最小可執行 **Python** 程式碼）

```
import numpy as np

class SharkToothSync:
    """
    對應論文 Section III-C, Algorithm 1 & 2
    可獨立針對單一連線即時執行的 Python 類別
    """
    def __init__(self, ci_ms=30.0, m_window=100, settling_sec=30.0):
        self.CI = ci_ms / 1000.0  # 轉為秒 (0.03 s)
        self.M = m_window
        self.settling_sec = settling_sec

        # 狀態變數
        self.delta_T_prev = None
        self.d2_T_buffer = []

        # 樞紐點與探針參數 (Eq. 21, 22)
        self.delta_T_0 = float('inf')
        self.t_P0 = 0.0
        self.gamma_0 = 0.0

        self.delta_T_1 = float('inf')
        self.t_P1 = 0.0
        self.gamma_1 = 1e-8

        self.delta_T_star_prev = 0.0

        # 遞迴 OLS 矩陣 (Eq. 16, 17)
        self.A = np.zeros((2, 2))
        self.b = np.zeros((2, 1))
        self.alpha = 0.0
        self.beta = 0.0

    def update(self, t_P, t_C):
        """
        輸入:
          t_P: 周邊本地時間戳 (秒)
          t_C: 中央主機時間戳 (秒)
```

```
        輸出:
          t_S_hat: 校正後的同步時間戳 (秒)
        """
        delta_T = t_C - t_P

        # 1. 計算二階差分與滑動標準差 (Eq. 5, 7)
        if self.delta_T_prev is not None:
            d2_T = delta_T - self.delta_T_prev
            self.d2_T_buffer.append(d2_T)
            if len(self.d2_T_buffer) > self.M:
                self.d2_T_buffer.pop(0)
            sigma_M = np.std(self.d2_T_buffer) if len(self.d2_T_buffer) > 1 else (self.CI / 2.0)
        else:
            sigma_M = self.CI / 2.0
        self.delta_T_prev = delta_T

        # 2. 計算品質加權係數 lambda_0 (Eq. 10)
        lambda_0 = min(self.CI / (2.0 * max(sigma_M, 1e-6)), 1.0)

        # 3. 尋找樞紐點或探針門檻過濾 (Algorithm 2)
        if delta_T < self.delta_T_0:
            # 發現全新下界點 (Eq. 11)
            self.delta_T_0 = delta_T
            self.t_P0 = t_P
            self.gamma_0 = 0.0
            self.gamma_1 = 1e-8
            # 觸發 30 秒沉降期重設 (Eq. 18)
            self.A.fill(0.0)
            self.b.fill(0.0)
            self.alpha = 0.0
            self.beta = 0.0
        else:
            # 與探針門檻比較 (Eq. 9)
            delta_T_thr = self.delta_T_1 + self.gamma_1 * (t_P - self.t_P1)
            if delta_T < delta_T_thr:
                # 合格候選點:自適應權重更新 (Eq. 12, 13)
                lam = 0.5 * lambda_0 if delta_T >= self.delta_T_star_prev else 0.5
                denom = max(t_P - self.t_P0, 1e-6)
                self.gamma_0 = (1.0 - lam) * self.gamma_0 + lam * ((delta_T - self.delta_T_0) / denom)
```

```
                self.gamma_1 = 1e-8
                self.delta_T_1 = delta_T
                self.t_P1 = t_P
            else:
                # 異常突波:門檻向上抬升 (Eq. 14)
                self.gamma_1 += 1e-6

        # 4. 計算校正下界延遲 (Eq. 8)
        delta_T_star = self.delta_T_0 + self.gamma_0 * (t_P - self.t_P0)
        self.delta_T_star_prev = delta_T_star

        # 5. 遞迴 OLS 更新 (Eq. 15–18)
        if t_P > (self.t_P0 + self.settling_sec):
            # 秩一增量累積
            dA = np.array([[1.0, -t_P], [-t_P, t_P**2]])
            db = np.array([[t_P * delta_T_star], [delta_T_star]])
            self.A += dA
            self.b += db

            det = self.A[0, 0] * self.A[1, 1] - self.A[0, 1] * self.A[1, 0]
            if abs(det) > 1e-9:
                res = np.dot(self.A, self.b) / det
                self.alpha = res[0, 0]
                self.beta = res[1, 0]

        # 6. 計算估計延遲與同步時間 (Eq. 19, 20)
        delta_T_hat = self.alpha * t_P + self.beta if abs(self.alpha) > 0 else delta_T_star
        t_S_hat = t_P + delta_T_hat

        return t_S_hat

# --- 單元測試與模擬突波驗證 ---
if __name__ == "__main__":
    sync = SharkToothSync(ci_ms=30.0)
    true_alpha = 15e-6  # +15 ppm 漂移
    true_beta = 1.05    # 1.05 秒初始相位差

    # 模擬 500 個封包 (10 Hz, 50 秒)
    for i in range(500):
```

```
        t_P = i * 0.1
        t_S = (1.0 + true_alpha) * t_P + true_beta

        # 正常延遲在 [0, 30ms] 內鋸齒波動
        base_delay = 0.015 + 0.015 * np.sin(2 * np.pi * i / 3.0)
        # 每 20 個封包插入一個 150ms 的重傳大突波
        noise = 0.15 if (i % 20 == 0) else 0.0

        t_C = t_S + base_delay + noise
        t_S_est = sync.update(t_P, t_C)

    print(f"真實漂移: {true_alpha:.2e}, 估計漂移: {sync.alpha:.2e}")
    print(f"第 500 步估計同步時間差 (誤差): {(t_S_est - t_S)*1000:.3f} ms")
```

關鍵超參數速查表(依據論文完全整理)

超參數名稱	符號 / 變數	數值設定	物理意義與功能
連線間隔	CI	30 ms	平衡吞吐量與延遲的 BLE 底層連線週期
滑動標準差視窗	M	100 樣本 (約 10 秒)	監控通訊品質與二階差分抖動
探針重設初始斜率	γ_1 (Init)	10^{-8}	重設探針門檻為接近水平線
探針步進遞增率	$\Delta\gamma_1$	10^{-6}	無合格點時向上抬頭,避免鎖死

超參數名稱	符號 / 變數	數值設定	物理意義與功能
自適應權重上限	λ_{max}	0.5	防止單一雜訊造成下界線過度擺動
沉降冷卻期時間	Settling Period	30 秒	樞紐點更新後等待下界線穩定之時間
基準方差門檻	$(CI/2)^2$	$225ms^2$	判定通訊是否進入擁塞的經驗分界點

第 8 部分：自我測驗

8.1 觀念與計算題（附詳解）

1. （觀念）為什麼在 **BLE** 連線模式下，重傳延遲只會是正向增加的離群值？

   【詳解】：封包若在當前連線事件中遺失，BLE 協定堆疊必須等待下一個連線事件（甚至再下一個）才能重傳。這導致延遲必然遞增 $1\times\text{CI}, 2\times\text{CI}, \dots$，物理上不可能提前到達。

2. （計算）若感測節點的晶體容差為 $\pm\ 20ppm$，連續採集 **2** 小時，未經同步的最大時鐘漂移量為多少毫秒？

   【詳解】：$2小時\ =\ 7,200秒$。最大時間偏差 $\Delta t\ =\ 7,200\ \times\ 20\ \times\ 10^{-6}\ =\ 0.144秒\ =\ 144ms$。這足以讓 1 kHz 的肌電訊號錯位 144 個採樣點！

3. （觀念）**SharkTooth** 演算法中的「樞紐點（**Pivot Point**）」物理意義為何？

   【詳解】：樞紐點是歷史觀測中時延 δT 的全域最小值。它代表該封包是以近乎零佇列排程、零重傳的極限物理速度抵達，最貼近真實的時鐘漂移底線，做為下界幾何切線的旋轉支點。

4. （計算）若 $CI\ =\ 30ms$，當前計算出滑動標準差 $\sigma_{100}\ =\ 45ms$，則品質係數 λ_0 為多少？

   【詳解】：代入 Eq. (10)：$\lambda_0\ =\ \min\left(\frac{30}{2\times45}, 1\right) =\ \min\left(\frac{30}{90}, 1\right) = \frac{1}{3} \approx\ 0.333$。

5. （觀念）在 **Eq. (13)** 中，當新點 $\delta T^{i} < \delta T_{*}^{i-1}$ 時，為什麼權重 λ^{i} 不受 λ_{0}^{i} 約束而直接強制設為 **0.5**？

   【詳解】：因為新點落於當前下界線下方，代表之前的下界線估算偏高了。為了防止演算法懸在空中，必須以最大步長（0.5）迅速拉回貼齊新點，確保「下界」名副其實。

6. （觀念）為什麼作者不把 **12** 個節點全部連到同一支 **USB Dongle**，而要拆成 **3** 支 **Dongle**？

   【詳解】：若單一 Dongle 承載 12 個節點且每個節點傳送 244 Bytes（需要 7.5 ms 事件長度），連線間隔必須拉長至 $12 \times 7.5 = 90ms$。這會導致每節點傳送週期必須大於 180 ms，吞吐量跌破 1.35 kB/s，無法滿足 1 kHz 肌電訊號（2 kB/s）的需求。

7. （觀念）**Ground-Truth** 驗證中，為何要將採樣訊號從 **50 Hz** 升取樣至 **10 kHz**？

   【詳解】：50 Hz 採樣的時間解析度僅有 $1/50 = 20ms$，無法評估亞毫秒級的同步誤差。升取樣至 10 kHz 可將時間解析度提升至 $0.1\text{ ms} = 100\ \mu\text{s}$，才能精確捕捉微小的相位差。

8. （計算）單通道 **1 kHz、16-bit** 的 **sEMG** 訊號，每秒產生的純數據吞吐量為多少 **kB/s**？

   【詳解】：每秒 1,000 個樣本，每個樣本 16 bits = 2 Bytes。總吞吐量 $= 1,000 \times 2B/s = 2,000B/s = 2.0kB/s$。

9. （觀念）為什麼樞紐點更新後要設置 **30** 秒的沉降冷卻期（**Settling Period**）？

   【詳解】：新樞紐點剛確立時，斜率 γ_{0} 正在自適應調整中，數值尚未平穩。若在此時將數據納入 OLS 回歸矩陣，會將過渡期的不穩定點永久累加進矩陣中，污染最終的長時時鐘估計。

10. （觀念）何謂互相關（**Cross-Correlation**）在此實驗中的模糊性半徑限制？

    【詳解】：對於週期性正弦波，互相關的最大可偵測時延限制在半個週期以內（5 Hz 正弦波週期 200 ms，半週期為 100 ms）。若同步誤差超過 100 ms，相位對齊會出現週期跳變歧義（Phase Ambiguity）。

8.2 延伸思考題

- 思考題 **1**：如果將連線間隔 **CI** 從 **30 ms** 縮短為 **7.5 ms**，**SharkTooth** 的同步表現會變好還是變差？

  【分析】：對於 2～4 個節點的小型網路，縮短 CI 會使鋸齒牙齒高度從 30 ms 降至 7.5 ms，下界更密集，同步精度會顯著提升（接近 0.1 ms）；但在 12 節點的高負載網路下，7.5 ms 會導致排程完全塞車，重傳碰撞率飆升，反而可能造成通訊斷線。

- 思考題 **2**：若將 **SharkTooth** 移植至微控制器韌體端（邊緣端），計算資源會遇到什麼瓶頸？

  【分析】：演算法的遞迴 OLS 與探針更新皆為 $O(1)$ 浮點運算，ARM Cortex-M4F 具有硬體 FPU，單次更新僅需數百個時脈週期（<10 μs），RAM 消耗小於 1 KB，運算資源完全無瓶頸，極具嵌入式邊緣移植可行性。

- 思考題 **3**：在運動員劇烈跑步時，**BLE** 封包遺失率飆升至 **50%** 以上，**SharkTooth** 會如何自處？

【分析】: 高掉包率會使滑動標準差 σ_{100} 爆表，導致 $\lambda_0 \to 0$。此時模型會智慧化進入「慣性滑行（Dead Reckoning）」狀態，暫停更新 γ_0，純粹依靠之前已收斂的高精度 α, β 繼續推估時間，展現出極強的抗斷線韌性。

第 9 部分：術語表

中文術語	英文全稱	簡稱	一句話定義	首次出現位置
無線人體感測網路	Wireless Body Sensor Network	WBSN	分佈於人體各處進行生理訊號與動作協同監測之無線節點系統。	p.46174
低功耗藍牙	Bluetooth Low Energy	BLE	專為物聯網低功耗設計之 2.4 GHz 雙模/單模無線通訊協定標準。	p.46174
連線間隔	Connection Interval	CI	BLE 連線模式中，中央與周邊節點進行排程資料交換之固定時間週期。	p.46175
現成商用元件	Commercial Off-The-Shelf	COTS	無需客製化晶片或修改專屬電路、可直接市購之標	p.46174

中文術語	英文全稱	簡稱	一句話定義	首次出現位置
			準商用電子產品。	
頻率容差	Frequency Tolerance	ppm	百萬分之一，石英晶體振盪器相對於標稱震盪頻率之最大物理偏差率。	p.46174
樞紐點	Pivot Point	—	歷史觀測時延差值之全域最小值，代表零重傳之理想幾何基準錨點。	p.46180
相對同步誤差	Relative Synchronization Error	RSE	演算法估計之兩節點相對時延與真實訊號相位時延之間的殘差。	p.46185
普通最小二乘法	Ordinary Least Squares	OLS	透過最小化殘差平方和以估計線性迴歸模型參數之經典統計方法。	p.46179
功能性電刺激	Functional Electrical Stimulation	FES	利用微弱電流刺激神經肌肉以恢復肢體運動功能之復健技術。	p.46176

中文術語	英文全稱	簡稱	一句話定義	首次出現位置
互相關	Cross-Correlation	CC	評估兩個訊號在不同相對時間位移下之相似程度以求解時滯之方法。	p.46184